UNIVERSIDAD POLITÉCNICA DE CHIAPAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

DOCUMENTO TÉCNICO DE ARQUITECTURA Y SOFTWARE

Aplicación Interactiva Móvil: Inquizitivo

Asignatura: Desarrollo de Aplicaciones Móviles

Docente: Mtro. Ali López Zúnun

Estudiante: Yesenia Guillén Moreno

Matrícula: 253408 | **Grupo:** 4° A

Fecha de Entrega: Octubre de 2026

Lugar: Suchiapa, Chiapas, México

Índice del Documento Técnico

1. **Descripción General del Proyecto**
2. **Requerimientos Funcionales y No Funcionales**
3. **Arquitectura del Sistema (MVVM y Patrones de Diseño)**
4. **Estructura y Jerarquía de Paquetes**
5. **Modelos de Datos y Paleta de Colores por Categoría**
6. **Diseño e Implementación de Pantallas (UI Layer)**
7. **Pruebas y Validación del Software**
8. **Conclusiones y Trabajo Futuro**

1. Descripción General del Proyecto

Inquizitivo es una aplicación móvil nativa desarrollada para la plataforma Android utilizando el toolkit moderno de interfaz de usuario **Jetpack Compose** y el lenguaje de programación **Kotlin**. La aplicación tiene como propósito ofrecer una experiencia de aprendizaje interactivo mediante cuestionarios tipo trivia organizados por categorías temáticas.

El sistema está diseñado bajo el patrón arquitectónico **MVVM (Model-View-ViewModel)**, lo que permite desacoplar la lógica de presentación de la interfaz de usuario de la gestión de datos. La aplicación opera de manera autónoma en el dispositivo, incorporando almacenamiento y datos semilla en memoria/local, con una experiencia fluida, diseño responsivo y navegación mediante componentes oficiales de Material Design 3.

2. Requerimientos Funcionales y No Funcionales

2.1 Requerimientos Funcionales

- **RF01 - Selección de Categorías:** La pantalla principal debe listar 5 categorías temáticas fijas (Programación, Base de Datos, Redes, Android y Cultura General), diferenciadas con colores distintivos.
- **RF02 - Presentación del Cuestionario:** Al seleccionar una categoría, se despliegan las preguntas correspondientes de una en una, junto a 4 opciones de selección única.
- **RF03 - Botón Flotante para Registro (FAB):** La pantalla de inicio incluirá un *Floating Action Button* en la esquina inferior derecha para permitir la navegación al formulario de creación de nuevas preguntas.
- **RF04 - Formulario de Preguntas Personalizadas:** Permite ingresar categoría, enunciado de la pregunta, cuatro opciones y marcar cuál es la respuesta correcta (índice 0 a 3). La pregunta creada se añade dinámicamente al catálogo de la categoría seleccionada.

- **RF05 - Cálculo de Resultados y Puntuación:** Al concluir la trivia, la app calcula la puntuación total (10 puntos por acierto), número de respuestas correctas e incorrectas y el porcentaje de efectividad.
- **RF06 - Módulo de Estadísticas:** Muestra la mejor puntuación histórica registrada, la cantidad total de cuestionarios completados y la categoría más jugada.

2.2 Requerimientos No Funcionales

- **RNF01 - Rendimiento:** Las pantallas y transiciones deben cargarse en un tiempo inferior a 2 segundos bajo condiciones normales de uso.
- **RNF02 - Mantenibilidad:** Código estructurado bajo MVVM que facilite la lectura y evaluación técnica en el ámbito académico.
- **RNF03 - Usabilidad:** Aplicación estricta de las guías de diseño de Material Design 3 para garantizar coherencia visual.

3. Arquitectura del Sistema (MVVM y Patrones de Diseño)

La aplicación implementa la arquitectura recomendada por Google para el desarrollo moderno en Android, dividiendo el sistema en tres capas principales:

Capa	Componentes	Responsabilidad Principal
Model (Modelo)	Category, Question, CustomQuestion, QuestionRepository	Define las estructuras de datos y gestiona el origen e inserción de la información de las preguntas.
ViewModel	QuizViewModel, AddQuestionViewModel, StatsViewModel	Procesa la lógica de negocio (evaluación de respuestas, cálculo de puntajes) y expone el estado mutable hacia la interfaz mediante StateFlow.
View (Vista)	HomeScreen, QuizScreen, ResultScreen,	Componentes Jetpack Compose que renderizan la UI

Capa	Componentes	Responsabilidad Principal
	AddQuestionScreen, StatsScreen	en función del estado recibido y notifican los eventos del usuario al ViewModel.

3.1 Aplicación del Patrón Factory Method

En el ámbito de la creación de objetos en Android, se utiliza el patrón **Factory Method** en dos niveles estratégicos:

1. **Inyección de ViewModels (ViewModelProvider.Factory):** Permite instanciar los ViewModels pasándoles dependencias requeridas en el constructor (como el QuestionRepository) sin comprometer el ciclo de vida gestionado por Android OS.
2. **Mapeo de Datos (QuestionFactory):** Unifica las preguntas semilla con las preguntas creadas por el usuario mediante un método de transformación que convierte estructuras crudas o de almacenamiento en instancias estándar de Question.

4. Estructura y Jerarquía de Paquetes

Para evitar conflictos de importación y mantener la claridad del proyecto en Android Studio, los archivos se organizan dentro del paquete raíz com.yess.inquizitivo siguiendo una estructura basada en responsabilidades:

```
com.yess.inquizitivo/
|
├── data/                                <-- CAPA MODELO Y REPOSITORIO
|   ├── model/
|   |   ├── Category.kt
|   |   ├── Question.kt
|   |   └── CustomQuestion.kt
|   └── repository/
|       └── QuestionRepository.kt
|
├── ui/                                  <-- CAPA DE PRESENTACIÓN (VISTAS Y
VIEWMODELS)
|   ├── screens/
|   |   ├── home/
```

```
|   |   |   └── HomeScreen.kt          <-- Vista de categorías con FAB
|   |   ├── quiz/
|   |   |   ├── QuizScreen.kt
|   |   |   └── QuizViewModel.kt
|   |   ├── results/
|   |   |   └── ResultScreen.kt
|   |   ├── stats/
|   |   |   ├── StatsScreen.kt
|   |   |   └── StatsViewModel.kt
|   |   └── add_question/
|   |       ├── AddQuestionScreen.kt
|   |       └── AddQuestionViewModel.kt
|   └── theme/
|       ├── Color.kt
|       ├── Theme.kt
|       └── Type.kt
|
├── navigation/                       <-- CAPA DE NAVEGACIÓN CENTRAL
|   └── AppNavigation.kt
|
└── MainActivity.kt                  <-- PUNTO DE ENTRADA DE LA APLICACIÓN
```

5. Modelos de Datos y Paleta de Colores por Categoría

Cada una de las 5 categorías cuenta con una representación cromática única asignada mediante valores de color hexadecimales para mejorar la usabilidad visual de la interfaz.

ID	Categoría	Color Representativo	Código Hexadecimal
1	Programación	Azul Material	#2196F3
2	Base de Datos	Verde Teal	#009688
3	Redes	Naranja Vívido	#FF9800

ID	Categoría	Color Representativo	Código Hexadecimal
4	Android	Verde Android	#4CAF50
5	Cultura General	Morado Intenso	#9C27B0

5.1 Definición de las Data Classes principales

```
// Category.kt
package com.yess.inquizitivo.data.model
import androidx.compose.ui.graphics.Color

data class Category(
    val id: Int,
    val name: String,
    val color: Color
)

// Question.kt
package com.yess.inquizitivo.data.model

data class Question(
    val id: Int,
    val categoryId: Int,
    val question: String,
    val options: List<String>,
    val correctAnswer: Int
)
```

6. Diseño e Implementación de Pantallas (UI Layer)

6.1 HomeScreen (Selección de Categorías y FAB)

Muestra las tarjetas con bordes redondeados y fondos de color según la categoría. Incluye un contenedor Scaffold con un FloatingActionButton (botón circular con icono +) fijado en la parte

inferior derecha.

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
    categories: List<Category>,
    onCategoryClick: (Category) -> Unit,
    onAddQuestionClick: () -> Unit
) {
    Scaffold(
        topBar = { TopAppBar(title = { Text("Inquizitivo - Categorías") })
},
        floatingActionButton = {
            FloatingActionButton(
                onClick = onAddQuestionClick,
                containerColor = MaterialTheme.colorScheme.primary
            ) {
                Icon(Icons.Default.Add, contentDescription = "Agregar
Pregunta", tint = Color.White)
            }
        }
    ) { padding ->
        LazyColumn(
            modifier = Modifier.padding(padding).padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            items(categories) { category ->
                Card(
                    onClick = { onCategoryClick(category) },
                    colors = CardDefaults.cardColors(containerColor =
category.color),
                    modifier = Modifier.fillMaxWidth().height(80.dp)
                ) {
                    Box(modifier = Modifier.fillMaxSize(), contentAlignment
= Alignment.Center) {
                        Text(category.name, color = Color.White, style =
MaterialTheme.typography.titleLarge)
                    }
                }
            }
        }
    }
}
```

6.2 QuizScreen (Desarrollo del Cuestionario)

Presenta las preguntas de forma individual, adaptando el color del encabezado a la categoría activa. Permite la selección interactiva de una opción y evalúa la respuesta al pulsar el botón "Siguiente" o "Finalizar".

6.3 ResultScreen (Evaluación de Rendimiento)

Muestra un resumen con la puntuación total acumulada, el recuento de aciertos y desaciertos, y botones para reiniciar la trivia o regresar a la pantalla principal.

6.4 AddQuestionScreen (Formulario de Registro)

Permite seleccionar la categoría mediante botones de radio (RadioButtons) e ingresar el texto de la pregunta junto con sus 4 opciones y el índice de la opción correcta.

7. Pruebas y Validación del Software

Para asegurar la estabilidad de la aplicación y el cumplimiento de los requerimientos de la asignatura, se llevaron a cabo las siguientes actividades de verificación:

- **Pruebas de Compilación y Renderizado:** Uso de las herramientas de previsualización (@Preview) en Android Studio para verificar la disposición estática de los componentes de Jetpack Compose en diferentes tamaños de pantalla.
- **Pruebas de Flujo de Navegación:** Confirmación del cambio de pantallas mediante el controlador de navegación y validación del comportamiento del botón flotante (FAB) al abrir el formulario de registro.
- **Pruebas de Lógica de Negocio:** Verificación del conteo de aciertos, cálculo de puntuaciones y adición dinámica de nuevas preguntas al banco en memoria.

8. Conclusiones

El desarrollo de la aplicación **Inquizitivo** permitió consolidar los principios de desarrollo nativo en Android mediante Jetpack Compose y la arquitectura MVVM. La separación de responsabilidades garantiza la escalabilidad del sistema y facilita futuras extensiones.

Líneas de Trabajo Futuro:

- Implementación de persistencia avanzada mediante la biblioteca **Room Database** para sincronizar preguntas personalizadas en SQLite.

- Integración de una API REST remota para descargar semanalmente nuevos bancos de preguntas.
- Inclusión de animaciones avanzadas en Jetpack Compose para retroalimentación visual al responder preguntas.

9. Repositorio

253408-yess/InQuizitivo: Cuestionario para estudiantes desarrollado en Kotlin